

Ghidra 기반 큐브샷 펌웨어 에뮬레이션 및 정적 분석 보강 프레임워크

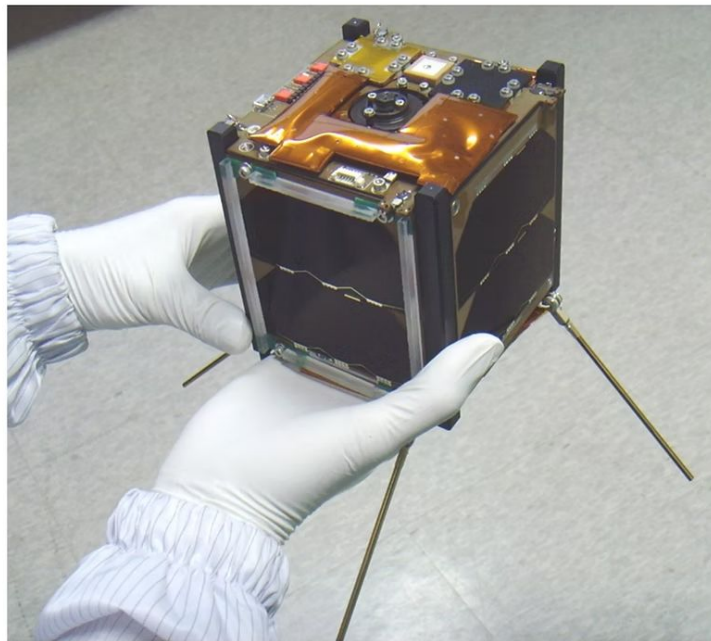
A Ghidra-based framework for CubeSat firmware emulation and static analysis augmentation

큐브샷이란?

CubeSat = Cube + Satellite

10cm 정육면체를 기본 단위(1U)로 사용하는
1U-12U 정도 크기의 소형 위성 규격

비교적 작은 크기와 낮은 개발 비용으로 인해 교육,
연구 등에서 널리 사용됨

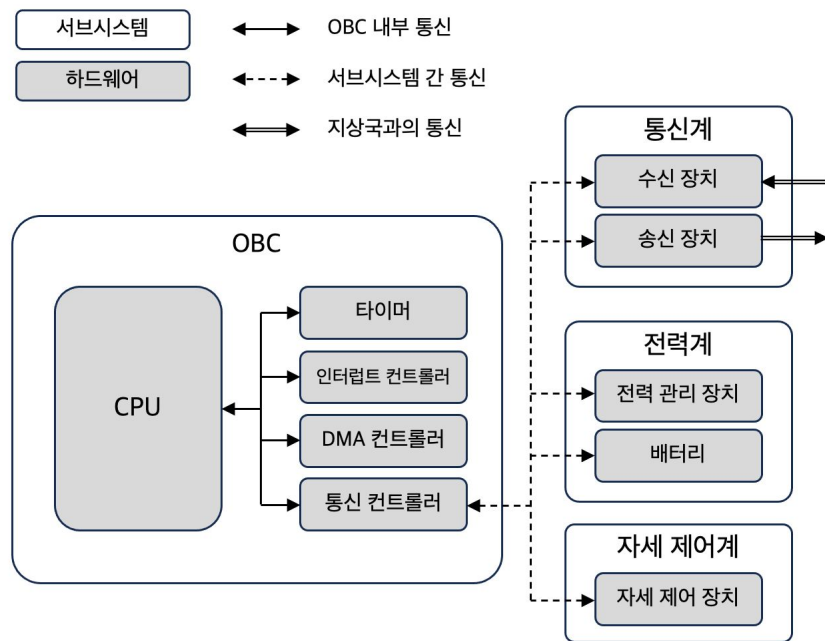


<https://www.smechaslab.kau.ac.kr/about-cubesat>

큐브셋의 일반적인 구조

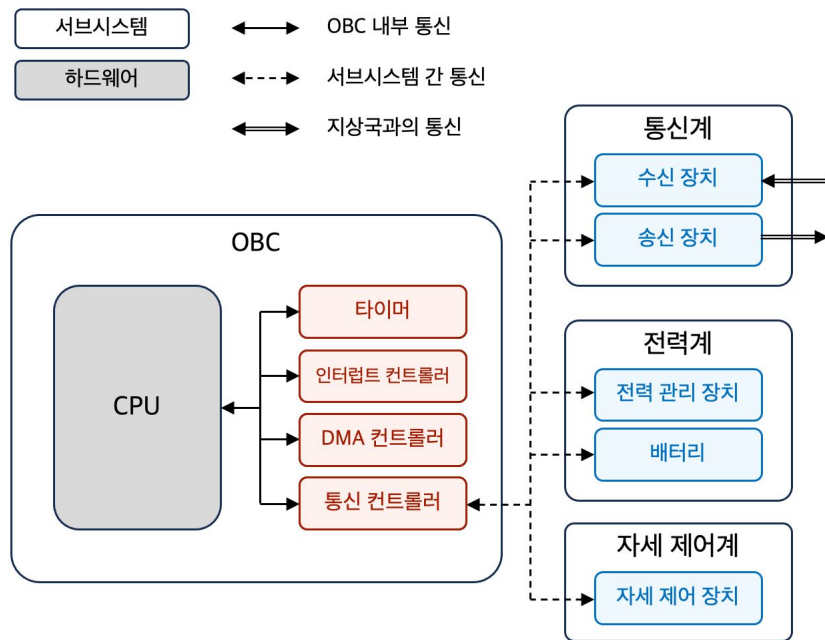
단일 보드가 아닌 **기능별 서브시스템**으로
나뉘어져 있음

- OBC (On-board computer)
 - 각 서브시스템 총괄 제어
- 통신계, 전력계, 자세 제어계
 - OBC와 연동되어 기능 수행



큐브셋의 일반적인 구조

- 주변장치
 - CPU를 제외한 모든 하드웨어
 - OBC 기준 내부 주변장치 및 외부 주변장치로 구분
- 각 주변장치는 큐브셋의 업무를 분담
 - 예시: 자세 제어 장치에서 위성 자세를 제어하기 위한 부동 소수점 연산들을 수행



큐브셋 OBC 펌웨어의 특징

1. 주변장치들에 대한 제어 흐름 의존성

구동 중 내/외부 주변장치와 지속적으로 값을 주고 받음

- 펌웨어 제어 흐름이 **주변장치와의 상호작용**에 영향을 받음
 - 예시: RANDEV OBC 펌웨어의 interrupt dispatch 함수

```
_int0:
```

```
...
```

```
MOV          R12, 0x0
```

```
MCALL
```

```
PC[->_get_interrupt_handler]
```

```
CP.W         R12, 0x0
```

```
MOV{NE}      PC, R12
```

```
RETE
```

..... _get_interrupt_handler 함수의 인자로 0을 설정

..... 핸들러 함수 주소를 받아 올

..... 핸들러 주소가 0이 아니면:

..... 해당 주소로 점프

큐브셋 OBC 펌웨어의 특징

1. 주변장치들에 대한 제어 흐름 의존성

구동 중 내/외부 주변장치와 지속적으로 값을 주고 받음

- 펌웨어 제어 흐름이 **주변장치와의 상호작용**에 영향을 받음
 - 예시: RANDEV OBC 펌웨어의 interrupt dispatch 함수

```
_int0:
    ...
    MOV          R12, 0x0
    MCALL
PC [-> _get_interrupt_handler]
    CP.W         R12, 0x0
    MOV{NE}     PC, R12
    RETE
```

인터럽트 컨트롤러에서
인터럽트 레벨에 맞는 핸들러
함수의 주소를 가져오는 함수

..... _get_interrupt_handler 함수의 인자로 0을 설정

..... 핸들러 함수 주소를 받아 올

..... 핸들러 주소가 0이 아니면:

..... 해당 주소로 점프

간접 점프 목적지가
주변장치에 의해 결정됨

큐브셋 OBC 펌웨어의 특징

2. 사전 검증의 중요성

큐브셋은 **발사 후 물리적 개입이 어려움**

따라서 발사 전 펌웨어 동작을 가능한 한 넓게 분석하고 잠재적 버그를 찾아야 함

테스팅

완전한 **실행 환경** 필요

실행된 경로에 대해서만
검증 가능

소스 코드 정적 분석

소스 코드 확보가 어려울 수 있음

컴파일/최적화 이후 최종
바이너리와 차이 있을 수 있음

바이너리 정적 분석

실행 환경 및 **소스 코드** 없이도
적용 가능

컴파일 및 최적화 영향까지 포함
가능

본 연구의 출발점

바이너리 정적 분석 및 Ghidra

바이너리 프로그램을 실제로 실행하지 않고 수행하는 분석

- 제어 흐름 그래프 생성
- 레지스터 값 추적
- 스택 포인터 추적

Ghidra: 바이너리 역공학 프레임워크

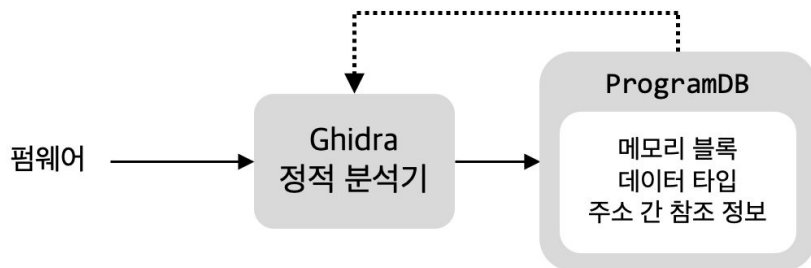
- 무료 오픈소스 도구
- 바이너리 정적 분석에 필요한 기능 제공
- 아키텍처별 명령어를 중간 표현인 P-code로 변환



바이너리 정적 분석 및 Ghidra

Ghidra는 원본 바이너리 및 분석 과정에서 생성한 정보를 **프로그램 데이터베이스**에서 관리

- 프로그램 데이터베이스는 추후 정적 분석에서 **재사용 및 업데이트**됨



References Editor					
Source					
8001f8f6 BR{eq} LAB_8001f8fc					
Operand	Destination	Label	Ref-Type	Primary?	Source
OP-0	8001f8fc	LAB_8001f8fc	CONDITIONAL_JUMP	☒	DEFAULT

큐브셋 OBC 펌웨어 정적 분석의 한계

주변장치 의존적인 제어 흐름을 파악하지 못함

```
_int0:
```

```
    MOV        R12, 0x0
```

```
    MCALL
```

```
PC [-> _get_interrupt_handler]
```

```
    CP.W       R12, 0x0
```

```
    MOV{NE}    PC, R12
```

```
    RETE
```

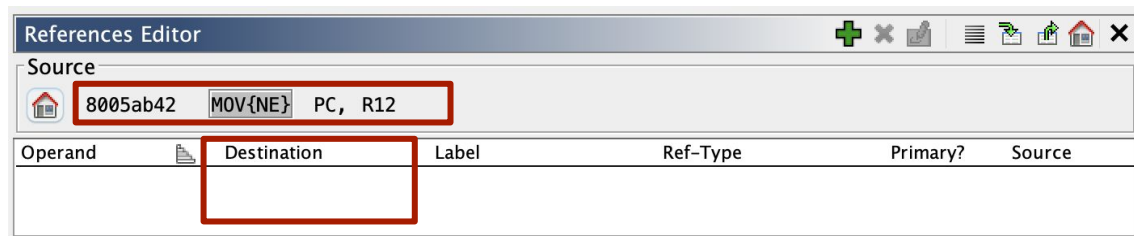
..... _get_interrupt_handler 함수의 인자로 0을 설정

..... 핸들러 함수 주소를 받아 옴

..... 핸들러 주소가 0이 아니라면:

..... 해당 주소로 점프

간접 점프 목적지가
주변장치에 의해 결정됨



큐브셋 OBC 펌웨어 정적 분석의 한계

주변장치 의존적인 제어 흐름을 파악하지 못함

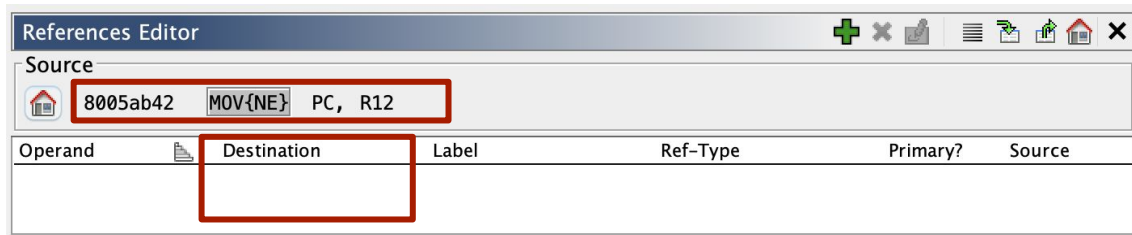
```
_int0:
```

```
MOV
```

```
R12, 0x0
```

..... _get_interrupt_handler 함수의 인자로 0을 설정

주변장치 의존적인 제어 흐름들이 프로그램 데이터베이스에서 누락되어 있음



아이디어: 에뮬레이션을 통한 프로그램 데이터베이스 보강

펌웨어를 에뮬레이터에서 직접 실행해
보고, 수집한 **동적 제어 흐름 로그**를 통해
기존 프로그램 데이터베이스를 보강

```
_int0:  
    MOV        R12, 0x0  
    MCALL  
PC[->_get_interrupt_handler]  
    CP.W       R12, 0x0  
    MOV{NE}    PC, R12  
    RETE
```

동적 제어 흐름 로그

8005ab42 -> 8002e0a4
8005ab42 -> 8002c538

References Editor						
Source						
8005ab42 MOV{NE} PC, R12						
Operand	Destination	Label	Ref-Type	Primary?	Source	
	8002e0a4					
	8002c538					

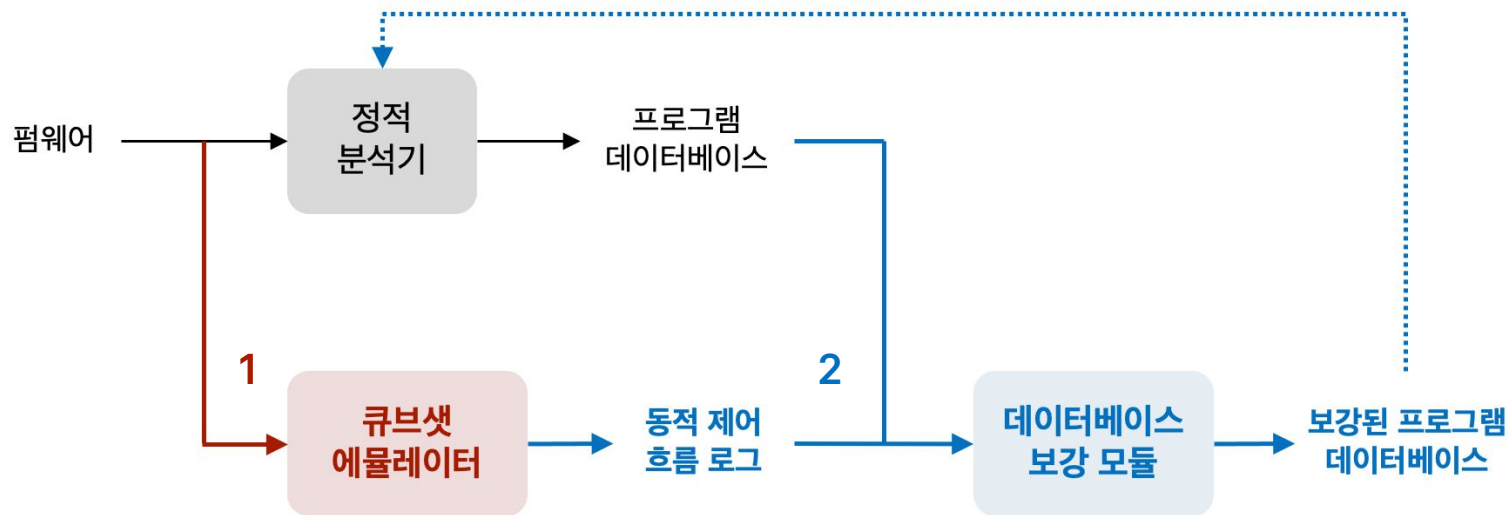
다양한 큐브셋 펌웨어를 실제 하드웨어 없이 실행할 수 있는

¹ 재사용 가능한 에뮬레이션 환경을 구성하고,

실행하면서 얻은 동적 제어 흐름 로그를 통해

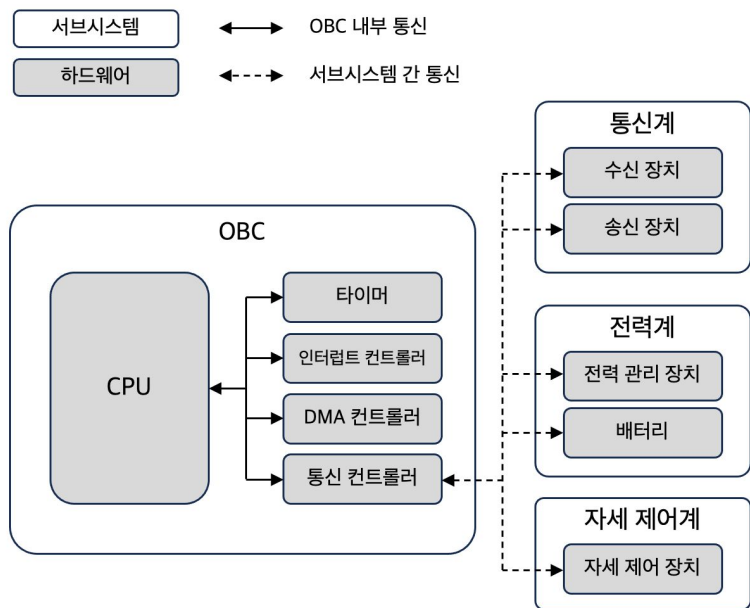
² 기존 프로그램 데이터베이스를 보강 및 추후 정적 분석 가능 범위를 확장

전체 프레임워크 개요



1. 큐브셋 에뮬레이터 구현

개요

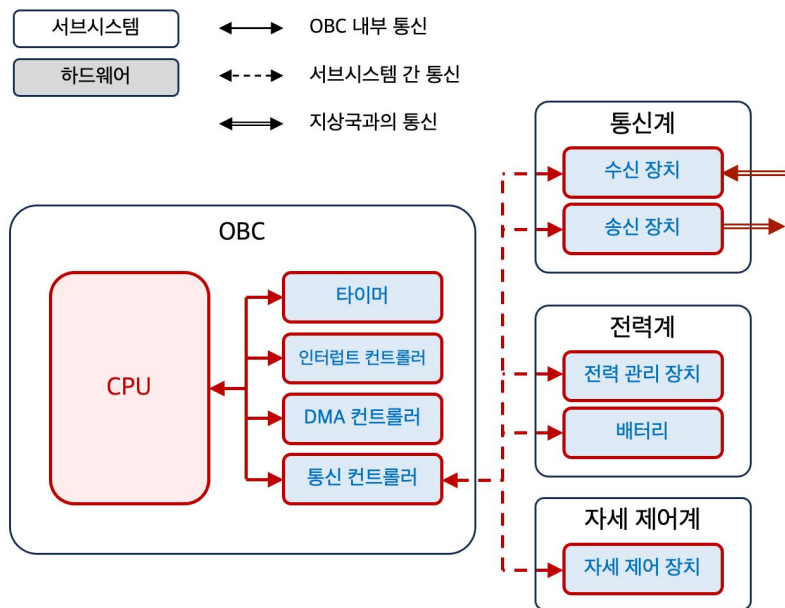


목표: 왼쪽 큐브셋 구조를
소프트웨어적으로 모방하는 것

요구사항: 다양한 큐브셋 펌웨어에 적용할
수 있는 **재사용 가능한 구조**

1. 큐브셋 에뮬레이터 구현

설계



재사용 가능 요소

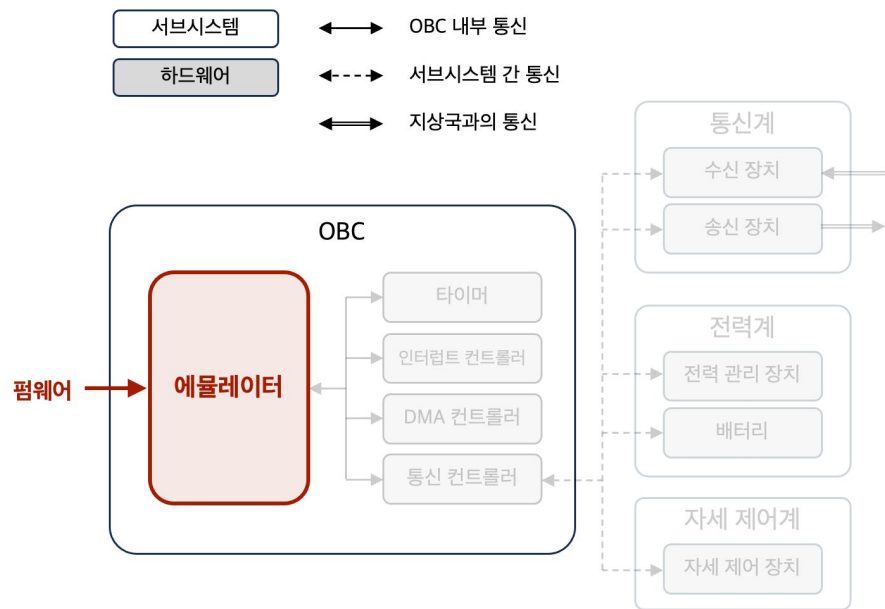
- CPU 에뮬레이션
- 통신 모델링
- 주변장치 모델 템플릿 구현

펌웨어 의존 요소

- 주변장치 세부 모델링
 - 내부 로직 및 레지스터 맵

1. 큐브셋 에뮬레이터 구현

CPU 에뮬레이션



펌웨어를 입력으로 받아 동작



임베디드 펌웨어
에뮬레이션에서 널리
쓰이는 도구

Ghidra 쪽과 별도의 연동
과정이 필요



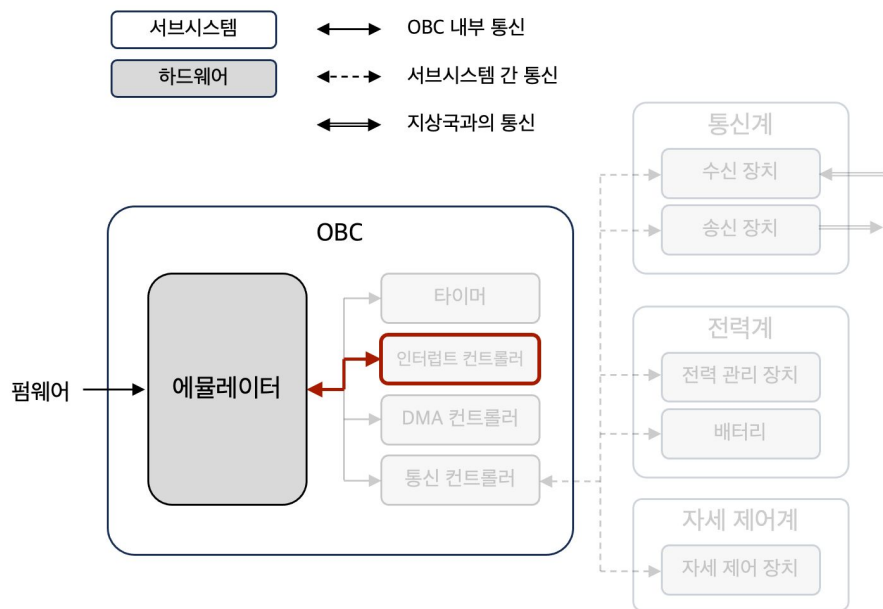
Ghidra script에서 지원하는
에뮬레이터

QEMU에 비해 많은 대상
아키텍처 지원

Ghidra script 기반 에뮬레이터로 결정

1. 큐브셋 에뮬레이터 구현

CPU 에뮬레이션

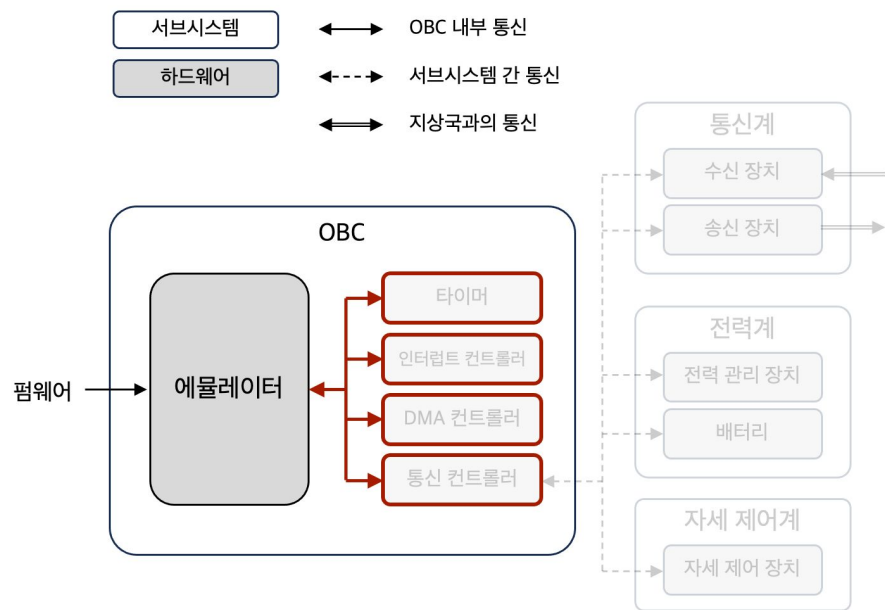


에뮬레이터와 인터럽트 컨트롤러 연결

실제 펌웨어에서 발생하는 **주기적 인터럽트**
및 **컨텍스트 스위칭** 재현 가능

1. 큐브셋 에뮬레이터 구현

OBC 내부 주변장치 모델 템플릿 및 통신 구현



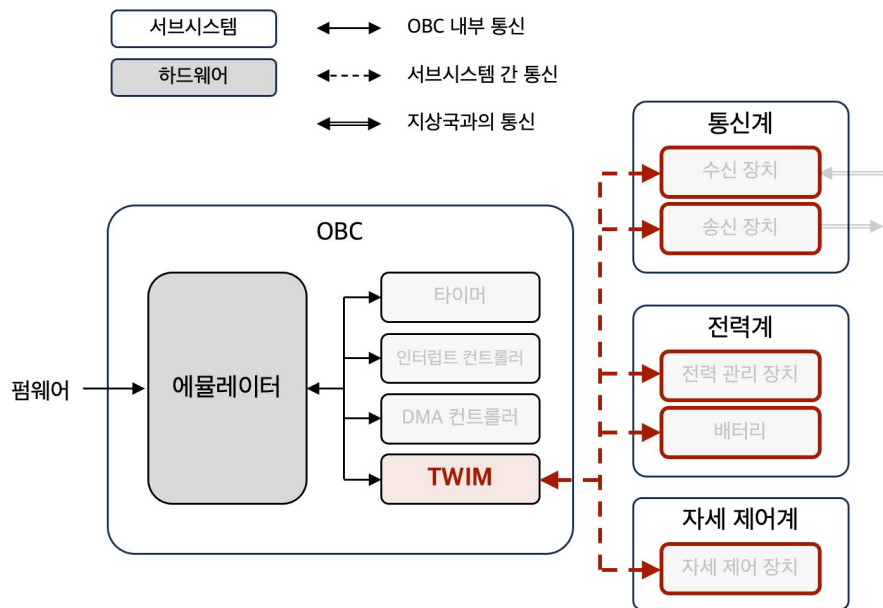
OBC 서브시스템에 있는 주변장치와 에뮬레이터 간 **MMIO 통신** 구현

- Memory-mapped I/O
 - 주변장치의 레지스터를 메모리 구역에 매핑하여 사용하는 것
 - 해당 영역에 값을 쓰면 실제 주변장치에서 부수효과를 수행

재사용 가능한 **내부 주변장치 모델 템플릿** 구현

1. 큐브셋 에뮬레이터 구현

외부 주변장치 모델 템플릿 및 통신 구현



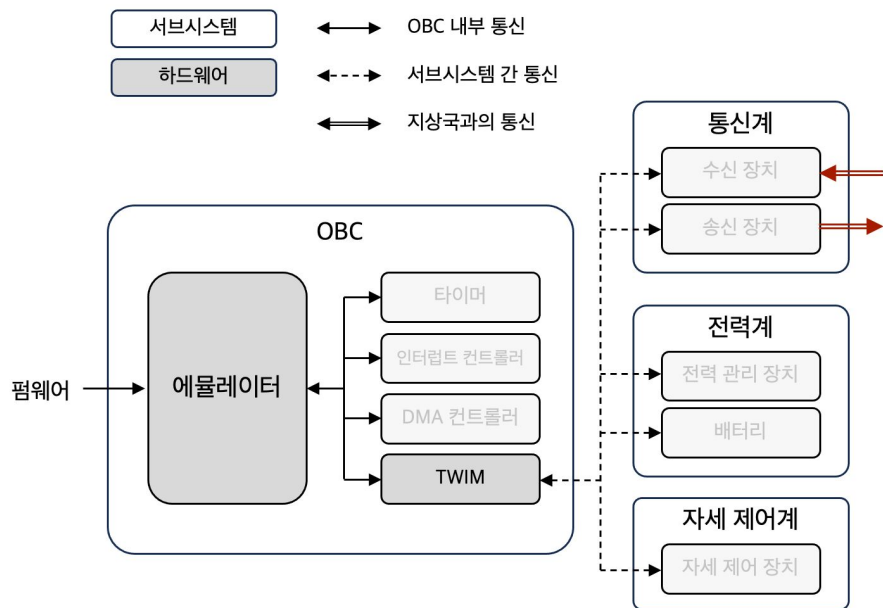
서브시스템 간 통신에서는 다양한 통신 인터페이스 사용 가능

현재 프레임워크에서는 I2C 통신을 전제 및 모델링

재사용 가능한 외부 주변장치 모델 템플릿 구현

1. 큐브셋 에뮬레이터 구현

지상국과의 통신 구현



위성과 지상국 간 통신에서는 일반적으로
무선 RF 통신을 사용

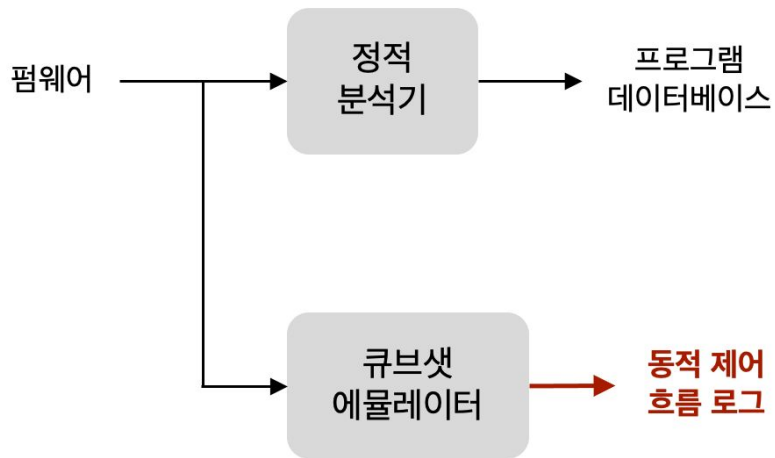
무선 RF 통신을 TCP 서버-클라이언트로
모사

- 지상국에서 명령을 받고 응답 가능

이후 **에뮬레이터 정합성 평가**에서 사용됨

2. 프로그램 데이터베이스 보강

동적 제어 흐름 로깅 구현



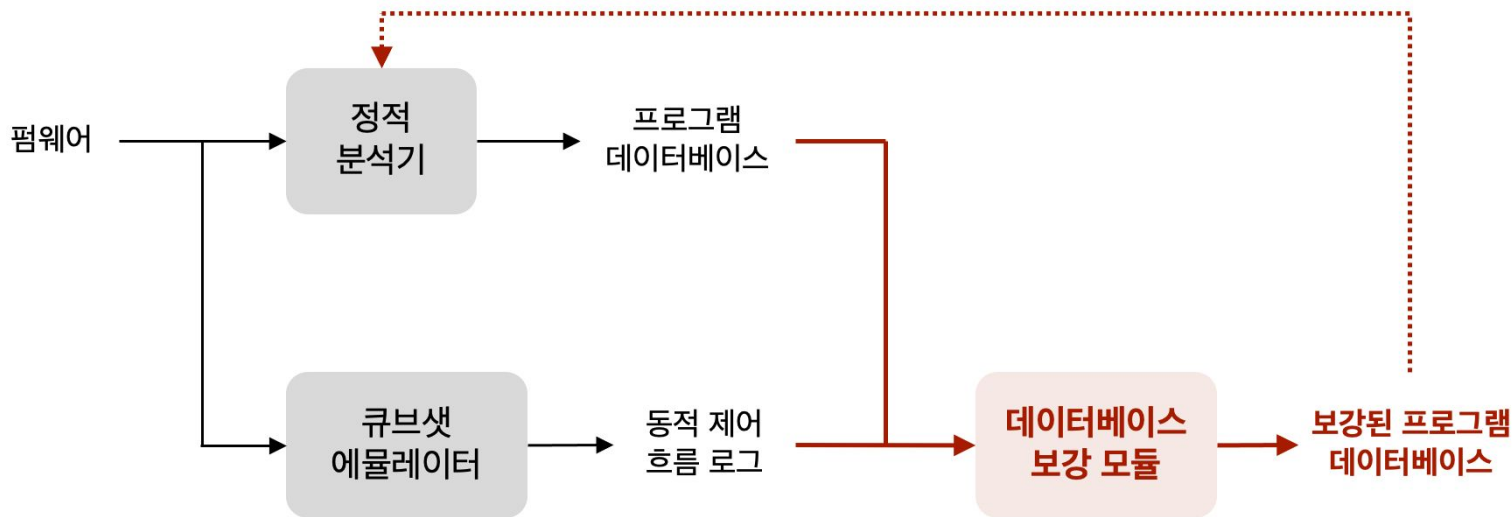
1. 큐브셋 에뮬레이터에서 펌웨어를 명령어 단위로 실행하면서
2. 실행한 명령어가 **간접 제어 흐름**을 만드는 명령어라면
 - Ghidra 내부 정보를 통해 **아키텍처** **중립적으로** 판단 가능
3. 해당 명령어 주소와 그 목적지를 기록

2. 프로그램 데이터베이스 보강

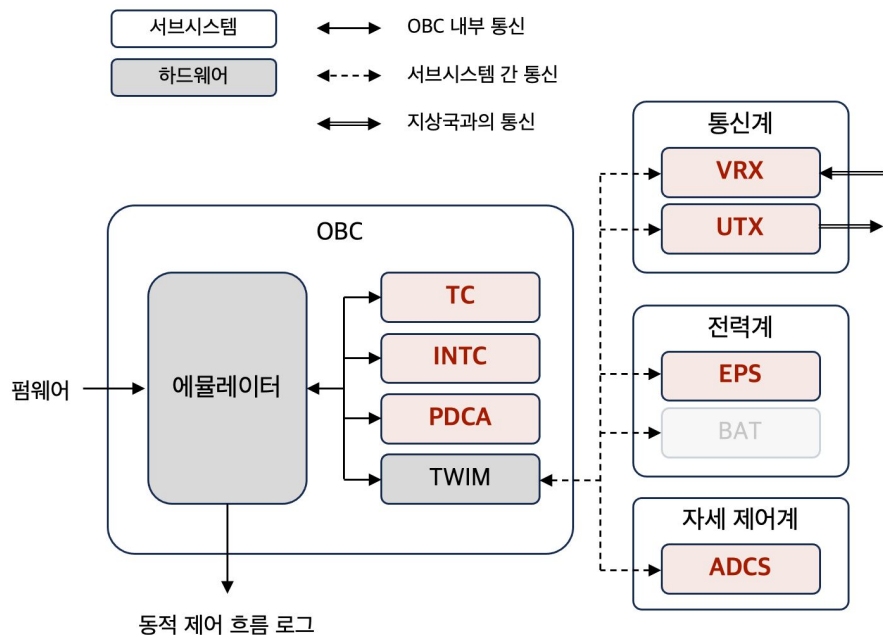
데이터베이스 보강 구현

프로그램 데이터베이스에 참조 추가

추가된 참조는 추후 분석에서 **기존 정적 분석 결과와 함께 사용됨**



케이스 스터디: RANDEV OBC 펌웨어



AVR32 기반 OBC 펌웨어

펌웨어 의존 요소 구현

- 장치별 세부 모델링
 - 내부 로직 및 레지스터 맵
 - 펌웨어 실행에 필수적인 주변장치 대상
- 사용자 연산 처리
 - 플랫폼 의존 요소

평가

1. 에뮬레이터가 지상국에서 보낸 명령에 대해 올바른 응답을 전송하는가?
2. 에뮬레이션을 통해 기존 정적 분석 결과를 보강하였는가?
3. 보강된 결과로 정적 분석을 시행했을 때 기존보다 더 넓은 범위를 분석할 수 있는가?

평가

1. 에뮬레이터가 지상국에서 보낸 명령에 대해 올바른 응답을 전송하는가?

평가 대상 명령: RANDEV의 서브시스템이 수행 가능한 모든 명령

각 명령에 대해 QEMU와 응답을 비교

- AVR32 전용 QEMU fork를 RANDEV에 맞게 직접 확장

장치	비교 대상 명령	일치	불일치
VRX/UTX	11	11	0
EPS	8	7	1
ADCS	9	9	0
HSTX	15	15	0
전체	43	42	1

비교 대상 명령 총 43개 중 42개에 대해서 일치 (97.7%)

- 불일치 사례: EPS (eps_get_status)
 - 시간 초과 발생

평가

2. 에뮬레이션을 통해 기존 정적 분석 결과를 보강하였는가?

평가 대상 위치: 총 58개

- Ghidra에서 디컴파일 결과 나온 간접 제어 흐름 위치 총 66개
- 이 중 모델링 대상이 아닌 콘솔 명령 계열 함수 소속 8개를 제외

지표	데이터베이스 보강 전	데이터베이스 보강 후
목적지 기록 위치	9/58 (15.5%)	22/58 (37.9%)
실행 중 목적지 관찰 위치	-	16/58 (27.6%)

분류	위치 수
미기록 위치의 목적지 신규 확인	13
기존 목적지 집합 확장	1
기존 목적지 재확인	2
전체	16

데이터베이스 보강 전후 목적지 파악 위치:
9개 -> 22개 (+13개)

- 목적지가 관찰되지 않은 이유 (36개)
 - 관련 주변장치가 구현되지 않았거나
 - 해당 위치의 실행 조건이 충족되지 않은 경우

평가

3. 보강된 결과로 정적 분석을 시행했을 때 기존보다 더 넓은 범위를 분석할 수 있는가?

보강 이전/이후 프로그램 데이터베이스에 대해 동일한 방식으로 호출 그래프 구축

main 및 태스크별 진입 지점에서부터 정적으로 도달 가능한 함수의 수 확인

평가 대상 함수: 953개

- 전체 함수 개수는 1460개이나 이전과 마찬가지로 콘솔 관련 함수들을 제외

지표	데이터베이스 보강 전	데이터베이스 보강 후
도달 가능 함수	744/953 (78.0%)	792/953 (83.1%)
신규 도달 가능 함수	-	48/209

도달 가능 함수: 744개 -> 792개 (+48)

추후 정적 분석에 반영되는 함수의
개수가 늘어남

논의

1. 다른 펌웨어에도 적용 가능한가

완전히 일반적인 프레임워크는 아니며 펌웨어 의존 요소들이 있음

펌웨어 의존 요소

- 주변장치 세부 구현
- 사용자 연산 처리 구현
- 기타 실행 환경 보정
 - 수동 역어셈블
 - 아키텍처 의존 레지스터 값 설정

재사용 가능 요소

- Ghidra 기반 에뮬레이션 구조
- 주변장치 모델링 템플릿
- 동적 제어 흐름 로그 출력 및 반영 로직

논의

2. 실행 속도 한계

구간	QEMU 시간(초)	Ghidra 시간(초)	속도 저하
명령 수신 준비 시간	1.003	12.015	12.0배
원격 측정 명령 처리 시간	5.941	199.924	33.7배
전체 실행 시간	6.943	211.938	30.5배

QEMU 기반 에뮬레이터와 비교하였을 때 확연히 느린 실행 속도

- 반복적인 고속 실행이 목적이 아님
 - 한 번의 실행만으로 데이터베이스 보강 가능
- 성능 위주의 상용 에뮬레이터가 아닌 **분석용 에뮬레이터**로는 활용 가능

다양한 큐브셋 펌웨어를 실제 하드웨어 없이 실행할 수 있는

¹ 재사용 가능한 에뮬레이션 환경을 구성하고,

실행하면서 얻은 동적 제어 흐름 로그를 통해

² 기존 프로그램 데이터베이스를 보강 및 추후 정적 분석 가능 범위를 확장

Backup Slides

큐브셋 OBC 펌웨어의 특징

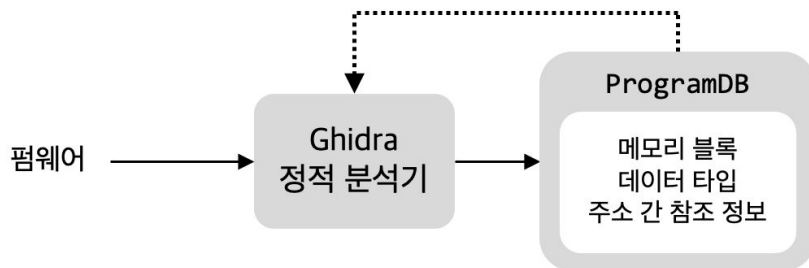
2. 사전 버그 탐지의 중요성

접근	한 줄 요약	장점	단점
테스팅	실행 기반 동작 확인	실제 동작 확인 가능	실행 환경/시나리오 필요, 실행 경로만 검증
소스 코드 정적 분석	소스 기반 구조/오류 분석	의미 정보 활용 가능	소스 코드 필요, 최종 바이너리와 차이 가능
바이너리 정적 분석	바이너리 기반 구조/제어 흐름 분석	소스 없이 시작 가능, 최종 산출물 직접 분석	주변장치 의존 제어 흐름 누락 가능

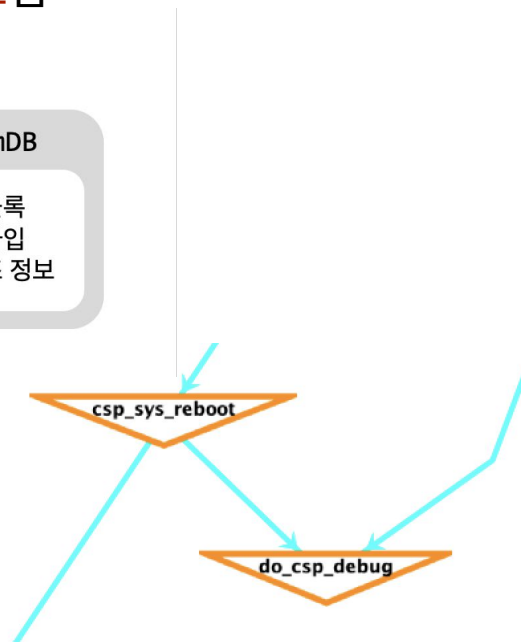
바이너리 정적 분석 및 Ghidra

Ghidra는 원본 바이너리 및 분석 과정에서 생성한 정보를 **프로그램 데이터베이스**에서 관리

- 프로그램 데이터베이스는 정적 분석에서 **재사용 및 업데이트**됨

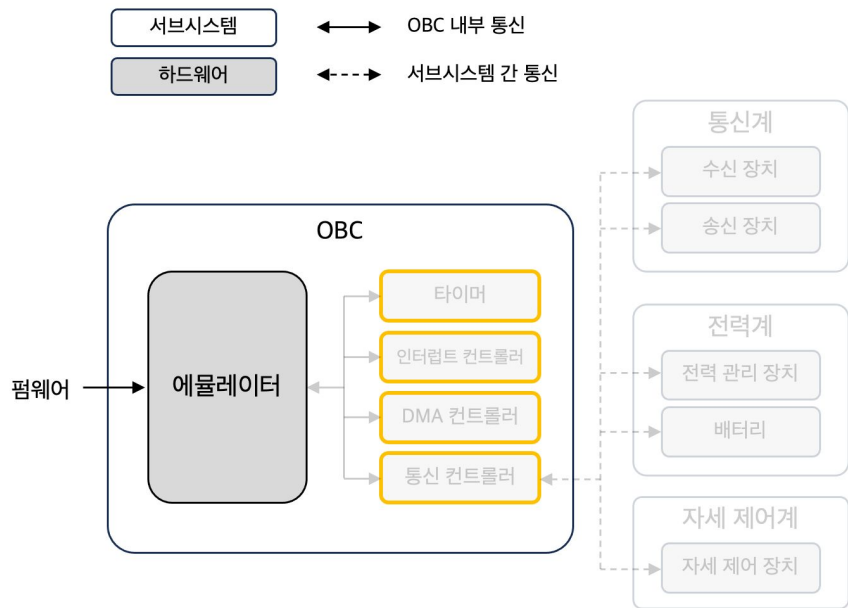


References Editor					
Source					
8001f908 MCALL PC[->do_csp_debug],					
Operand	Destination	Label	Ref-Type	Primary?	Source
MNEMONIC	80020350	do_csp_debug	COMPUTED_CALL	☒	ANALYSIS



큐브셋 에뮬레이터 구현

MMIO 장치 모델 템플릿 구현



```
public MmioDevice(DeviceManager deviceManager, long base,
String name, int group, long size)
```

```
protected MmioRegion(MmioDevice owner, int baseOffset, int
size)
```

MMIODevice

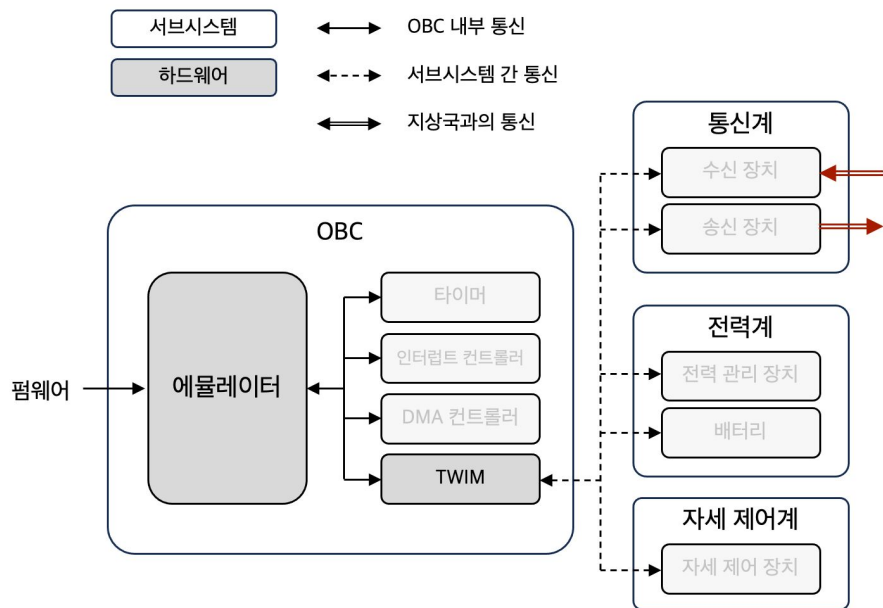
- MMIO 메모리 주소 및 범위
- 장치 이름
- 인터럽트 그룹

MMIORegion

- 최상위 소유자 (MMIODevice)
- 장치 내 메모리 주소 오프셋 및 범위

1. 큐브셋 에뮬레이터 구현

지상국과의 통신 구현



위성과 지상국 간 통신에서는 일반적으로
무선 RF 통신을 사용

무선 RF 통신을 TCP 서버-클라이언트로
모사

RF가 뭔지? TCP가 뭔지?